Python Common Services SILKWAVE Transport

Table of Contents

Copyright Notice . . . 1
Change History . . . 1
1. Introduction . . . 2
2. SILKWAVE Client . . . 3
  2.1. Connecting . . . 3
  2.2. Configuration . . . 3
    2.2.1. SSL Connections . . . 4
    2.2.2. HTTPS File Transfers . . . 5
    2.2.3. Overriding the SILKWAVE hostname . . . 5
    2.2.4. JSON Configuration . . . 6
    2.2.5. CLI Options . . . 6
3. Ping . . . 10
  3.1. Responses . . . 10
  3.2. Requests . . . 10
4. Utilities . . . 11
  4.1. sw-ping . . . 11

Copyright Notice

Change History

1.0	Initial Version - Split from pycs-core 4.5.1
1.1	Simplified RTP streaming interface
1.2	Allow STOMP connection over SSL
2.0	Transport abstraction

1. Introduction

This package provides an implementation of a pycs.transport.Transport, allowing connection to a SILKWAVE Hub, sending and receiving of messages, and file transfer and streaming functionality.

2. SILKWAVE Client

The `pycs.silkwave` package contains classes for connecting to and communicating with a SILKWAVE hub in an `asyncio` environment.

2.1. Connecting

In its simplest form, the connection will attempt to connect to a SILKWAVE hub on localhost port 61623 using the STOMP protocol. The SilkwaveClient class acts as an `AsyncContextManager` for use in an `async with` statement.

```
import asyncio
from pycs.silkwave import SilkwaveClient, SilkwaveConfig



async def example(config: SilkwaveConfig):
    async with SilkwaveClient(config) as client:
        ...

if __name__ == '__main__':
    asyncio.run(example(SilkwaveConfig()))
```

2.2. Configuration

The `SilkwaveConfig` object passed to the client allows configuration of the SILKWAVE connection. Some of the common options are shown in the example below - see the `pycs.silkwave.config` package for details of further options.

```
from pycs.silkwave import SilkwaveClient, SilkwaveConfig, ConnectionConfig

async def example():
    async with SilkwaveClient(
                    SilkwaveConfig(
                        name='Example Client',
                        connection=ConnectionConfig(
                            host='silkwave.server',
                            port=61623,
                            username='silkwave-user',
                            password='letmein',
                        ),
                    ),
                ) as sw:
        ...
```

2.2.1. SSL Connections

The `ConnectionConfig` object has an `ssl` field to configure an SSL context for the connection. Setting this to `True` will cause the SILKWAVE connection to communicate over SSL, using the default settings for the system.

```
from pycs.silkwave import SilkwaveClient, SilkwaveConfig, ConnectionConfig

async def example():
    async with SilkwaveClient(
                    SilkwaveConfig(
                        name='Example Client',
                        connection=ConnectionConfig(
                            host='silkwave.server',
                            port=61622,
                            username='silkwave-user',
                            password='letmein',
                            ssl=True,
                        ),
                    ),
                ) as sw:
        ...
```

Client certificates and keys, and server certificate authorities can be supplied by passing an SslConfig object to the `ssl` field.

```
from pycs.silkwave import SilkwaveClient, SilkwaveConfig, ConnectionConfig

async def example():
    async with SilkwaveClient(
                    SilkwaveConfig(
                        name='Example Client',
                        connection=ConnectionConfig(
                            host='silkwave.server',
                            port=61622,
                            username='silkwave-user',
                            password='letmein',
                            ssl=SslConfig(
                                certificate=Path('cert.pem'),
                                passphrase='certpass',
                                ca=Path('ca.pem')
                            ),
                        ),
                    ),
                ) as sw:
        ...
```

2.2.2. HTTPS File Transfers

SILKWAVE uses HTTP(S) for file transfers. By default these will use the same SSL settings as the main connection to configure HTTPS. If a different configuration is required, the `files` property on the `SilkwaveConfig` may be used. Setting the `https` field to 'True' (the default) will use the same configuration as the connection SSL. Setting to `False` will force HTTP, and passing an SslConfig object will override the configuration completely.

```
from pycs.silkwave import SilkwaveClient, SilkwaveConfig, ConnectionConfig, FileTransferConfig

async def example():
    async with SilkwaveClient(
                    SilkwaveConfig(
                        name='Example Client',
                        connection=ConnectionConfig(
                            host='silkwave.server',
                            port=61622,
                            username='silkwave-user',
                            password='letmein',
                            ssl=SslConfig(
                                certificate=Path('cert.pem'),
                                passphrase='certpass',
                                ca=Path('ca.pem')
                            ),
                        ),
                        files=FileTransferConfig(
                            https=False,
                        )
                    ),
                ) as sw:
        ...
```

2.2.3. Overriding the SILKWAVE hostname

When running in containerised environments or behind any sort of NAT, it is sometimes useful to override the hostname that SILKWAVE uses to advertise where file transfers or streams should be sent to.

This can be done using the `force_hub_hostname` property on the `files` and `streams` configurations.

```
from pycs.silkwave import SilkwaveClient, SilkwaveConfig, FileTransferConfig, StreamConfig

async def example():
    async with SilkwaveClient(
                     SilkwaveConfig(
                         files=FileTransferConfig(
                             force_hub_hostname='silkwave.host',
                         ),
                         streams=StreamConfig(
                             force_hub_hostname='silkwave.host',
```

```
                    )
                ),
            ) as sw:
        ...
```

2.2.4. JSON Configuration

All of the configuration classes are Pydantic models, and so can be populated from a JSON configuration file. See the Pydantic documentation for more details.

```
from pycs.silkwave import SilkwaveClient, SilkwaveConfig
from pathlib import Path

async def example():
    config = SilkwaveConfig.model_validate_json(Path('config.json').read_text())
    async with SilkwaveClient(config) as sw:
        ...
```

2.2.5. CLI Options

An `ApplicationSettings` class is provided to assist with supplying config data as a command line parameters. This class utilises Pydantic Settings to read configurations from the command line, environment variables and other sources.

This class should be used as the base class of a configuration object with your application options. It provides additional options to allow supplying a JSON config file, printing the schema for the config file, and specifiying a log configuration file.

```
from pycs.silkwave import SilkwaveClient, SilkwaveConfig, ApplicationSettings
from pydantic import BaseModel


class AppConfig(BaseModel):
    name: str = "example"

class ExampleSettings(ApplicationSettings):
    silkwave: SilkwaveConfig = SilkwaveConfig()
    app: AppConfig = AppConfig()

async def run():
    config = ExampleSettings()
    async with SilkwaveClient(config.silkwave) as sw:
        ...

if __name__ == "__main__":
    try:
        asyncio.run(run())
    except KeyboardInterrupt:
```

```
pass
```

```
> python -m pycs.silkwave.example_client --help
usage: example_client.py [-h] [--config Path] [--config-schema | --no-config-schema] [--log-config Path] [--silkwave
[JSON]] [--silkwave.name str] [--silkwave.security [JSON]] [--silkwave.security.classification {U,R,C,S,TS}] [--
silkwave.security.sci-controls str] [--silkwave.security.releaseable-to str]
                         [--silkwave.security.access-controls str] [--silkwave.security.owner-producer str] [--
silkwave.security.creator-organization str] [--silkwave.security.classified-by str] [--silkwave.security.classification-
reason str] [--silkwave.security.derived-from str]
                         [--silkwave.security.declass-date str] [--silkwave.security.declass-exception str] [--
silkwave.security.auth-id str] [--silkwave.security.alt-label str] [--silkwave.connection [JSON]] [--
silkwave.connection.host str] [--silkwave.connection.port int]
                         [--silkwave.connection.username str] [--silkwave.connection.password str] [--
silkwave.connection.ssl [{JSON,bool}]] [--silkwave.connection.ssl.key Path] [--silkwave.connection.ssl.passphrase str]
[--silkwave.connection.ssl.certificate Path] [--silkwave.connection.ssl.ca Path]
                         [--silkwave.connection.ssl.ca-dir Path] [--silkwave.files [JSON]] [--silkwave.files.force-hub-
hostname str] [--silkwave.files.https [{JSON,bool}]] [--silkwave.files.https.key Path] [--silkwave.files.https.passphrase
str] [--silkwave.files.https.certificate Path]
                         [--silkwave.files.https.ca Path] [--silkwave.files.https.ca-dir Path] [--silkwave.streams
[JSON]] [--silkwave.streams.force-hub-hostname str] [--app [JSON]] [--app.name str]

options:
  -h, --help            show this help message and exit
  --config Path         Load configuration from YAML/JSON file (default: null)
  --config-schema, --no-config-schema
                        Print configuration schema and exit (default: False)
  --log-config Path     Load logging configuration from YAML/JSON file - see
https://docs.python.org/3/library/logging.config.html#logging-config-dictschema (default: null)

silkwave options:
  --silkwave [JSON]     set silkwave from JSON string (default: {})
  --silkwave.name str   Name to identify this client on the SILKWAVE network (default: None)

silkwave.security options:
  The security marking to use on the envelope of all messages sent to the SILKWAVE hub where a more specific marking is
not supplied by the sender.

  --silkwave.security [JSON]
                        set silkwave.security from JSON string (default: {})
  --silkwave.security.classification {U,R,C,S,TS}
                        (default: U)
  --silkwave.security.sci-controls str
                        (default: None)
  --silkwave.security.releaseable-to str
                        (default: None)
  --silkwave.security.access-controls str
                        (default: None)
  --silkwave.security.owner-producer str
                        (default: None)
  --silkwave.security.creator-organization str
                        (default: None)
  --silkwave.security.classified-by str
                        (default: None)
  --silkwave.security.classification-reason str
                        (default: None)
  --silkwave.security.derived-from str
                        (default: None)
  --silkwave.security.declass-date str
                        (default: None)
  --silkwave.security.declass-exception str
                        (default: None)
  --silkwave.security.auth-id str
                        (default: None)
  --silkwave.security.alt-label str
```

```
                        (default: None)

silkwave.connection options:
  Configuration for the SILKWAVE connection

  --silkwave.connection [JSON]
                        set silkwave.connection from JSON string (default: {})
  --silkwave.connection.host str
                        Hostname of the SILKWAVE hub to connect to (default: localhost)
  --silkwave.connection.port int
                        Port number of the SILKWAVE hub to connect to. (default: 61623)
  --silkwave.connection.username str
                        Username to authenticate with SILKWAVE hub (default: client)
  --silkwave.connection.password str
                        Password to authenticate with SILKWAVE hub (default: manager)

silkwave.connection.ssl options:
  SSL configuration for SILKWAVE connections
  Set to 'false' to turn off SSL
  Set to 'true' to use a default SSL configuration

  --silkwave.connection.ssl [{JSON,bool}]
                        set silkwave.connection.ssl from JSON string (default: {})
  --silkwave.connection.ssl.key Path
                        PEM file containing SSL private key (default: null)
  --silkwave.connection.ssl.passphrase str
                        Passphrase to decrypt SSL private key (default: null)
  --silkwave.connection.ssl.certificate Path
                        PEM file containing SSL certificate, or certificate and key (default: null)
  --silkwave.connection.ssl.ca Path
                        PEM file containing trusted CA certificates (default: null)
  --silkwave.connection.ssl.ca-dir Path
                        Path to OpenSSL truststore directory containing CA certificates (default: null)

silkwave.files options:
  Configuration for file transfers

  --silkwave.files [JSON]
                        set silkwave.files from JSON string (default: {})
  --silkwave.files.force-hub-hostname str
                        Override hostname of the local SILKWAVE hub (default: None)

silkwave.files.https options:
  SSL configuration for HTTPS connections
  Set to 'false' to use HTTP
  Set to 'true' to use same settings as SILKWAVE connection

  --silkwave.files.https [{JSON,bool}]
                        set silkwave.files.https from JSON string (default: {})
  --silkwave.files.https.key Path
                        PEM file containing SSL private key (default: null)
  --silkwave.files.https.passphrase str
                        Passphrase to decrypt SSL private key (default: null)
  --silkwave.files.https.certificate Path
                        PEM file containing SSL certificate, or certificate and key (default: null)
  --silkwave.files.https.ca Path
                        PEM file containing trusted CA certificates (default: null)
  --silkwave.files.https.ca-dir Path
                        Path to OpenSSL truststore directory containing CA certificates (default: null)

silkwave.streams options:
  Configuration for streams

  --silkwave.streams [JSON]
                        set silkwave.streams from JSON string (default: {})
  --silkwave.streams.force-hub-hostname str
```

```
                        Override hostname of the local SILKWAVE hub (default: None)

app options:
  --app [JSON]          set app from JSON string (default: {})
  --app.name str        (default: example)
```

3. Ping

3.1. Responses

The client will automatically respond to `PingRequest` messages from a SILKWAVE hub or from another client with an appropriate `PingResponse` message.

3.2. Requests

To send a `PingRequest` from the client and handle the response, the `ping()` function can be called, which returns an `AsyncIterator` to iterate over the responses.

There are three modes that the pinger can use `PingMode.Normal`, `PingMode.Hub` and `PingMode.TraceRoute`. These correspond to the three modes defined in the SILKWAVE IDD, Section 3.5.8. In addition a `hub_only` flag can be set which stops the ping request at the end client's parent hub and does not ping the client itself. This can be useful for checking that there is a route to a client that does not repspond to `PingRequest` messages.

```
from pycs.silkwave import *

async def ping_client(target_host: str, config: SilkwaveConfig):
    async with SilkwaveClient(config) as client:
        async for responder, hops in client.ping(target_host, mode=PingMode.TraceRoute):
            print(f'{responder} : {hops}')
```

4. Utilities

4.1. sw-ping

A command line tool `sw-ping` is provided to utilise the ping functionality built into the `SilkwaveClient`. This enables pings and trace routes of SILKWAVE hubs and clients. To ping a hub, its `:networkstatus` address should be used.

ping a client

```
(venv)$ sw-ping net:silkwave.example.hub:012345
net:silkwave.example.hub1:012345 ok
```

ping a client and its parent hub

```
(venv)$ sw-ping --hub net:silkwave.example.hub:012345
net:silkwave.example.hub1:networkstatus ok
net:silkwave.example.hub1:012345 ok
```

ping a hub

```
(venv)$ sw-ping net:silkwave.example.hub:networkstatus
net:silkwave.example.hub1:networkstatus ok
```

trace route to a client

```
(venv)$ sw-ping net:silkwave.example.hub1:012345 --trace
net:silkwave.example.hub3:networkstatus ok (1 hop)
net:silkwave.example.hub2:networkstatus ok (2 hops)
net:silkwave.example.hub1:networkstatus ok (3 hops)
net:silkwave.example.hub1:012345 ok
```